

Neural Network and Deep Learning Project 1 Report

姓名：于翔 学号：23307130202

2026 年 5 月 7 日

摘要

本项目研究 MNIST 手写数字分类任务。根据课程要求，我首先基于 NumPy 实现 MLP baseline，包括线性层前向/反向传播、ReLU、softmax cross entropy、SGD 训练和学习曲线记录；随后手写实现二维卷积算子，并构建一个小型两层 CNN，在相同数据划分和相近训练设置下与 MLP 进行比较。完成 Part A 和 Part B 后，我选择 optimization 与 regularization 两个额外方向进行探索，系统比较 SGD、Momentum、Adam、learning rate scheduling、weight decay、dropout 和组合 recipe。此外，我进一步进行 detailed visualization，包括 confusion matrix、错分类样本、MLP 权重、CNN 卷积核以及两层卷积中间 feature maps。

实验结果显示，MLP baseline 的 test accuracy 为 0.95440；手写 CNN baseline 提升到 0.97790；最佳训练 recipe 进一步达到 0.98560。错误分析显示，剩余错误主要集中在形状相似的数字对，例如 4/9、5/3、7/2 和 8/0。卷积中间特征可视化表明，浅层卷积确实会强化笔画和边缘区域，但第二层卷积更像是在组合局部 stroke patterns，而不是直接输出人眼意义上的完整轮廓图。

1 Project Requirements and Experimental Setup

1.1 Requirements

课程文档要求本项目只使用给定的 MNIST 数据集，并且不能调用 PyTorch 等深度学习框架中直接完成任务的神经网络模块。本项目的核心目标不是追求最高精度，而是完成可运行、可解释、可比较的神经网络实现。报告需要覆盖以下内容：

- Part A: MLP baseline，包括线性层、softmax cross entropy、训练/验证表现和 learning curve。
- Part B: 手写 conv2D，构建 CNN，并与 MLP 在合理公平设置下比较。
- Part C: 选择两个额外方向，并对每个方向解释改动、基线、结果和结论。
- Main results table、detailed visualization 和 discussion。

本文选择的两个 Part C 方向是：

1. Optimization: 从训练算法和学习率策略角度探索模型效果改善。
2. Regularization: 从 weight decay、dropout 和 early stopping 角度分析泛化表现。

1.2 Dataset and Split

MNIST 数据集包含 60000 张训练图像和 10000 张测试图像。每张图像为 28×28 灰度图，标签属于 $\{0, \dots, 9\}$ 。令样本为 (x_i, y_i) ，其中

$$x_i \in [0, 1]^{28 \times 28}, \quad y_i \in \{0, 1, \dots, 9\}.$$

所有输入像素由 $[0, 255]$ 归一化到 $[0, 1]$ 。实验固定随机种子为 309，并从原始训练集划分 10000 张作为 validation set，剩余 50000 张作为 training set。测试集只用于最终 test accuracy 和错误分析。

1.3 Code Organization and Reproducibility

主要实现位于 `codes/` 目录下。核心代码映射如下：

File	Main responsibility
<code>mynn/op.py</code>	Core layers and loss: linear, convolution, activation, pooling, dropout, softmax cross entropy
<code>mynn/models.py</code>	Model_MLP, Model_CNN, forward/backward, save/load
<code>mynn/optimizer.py</code>	SGD, MomentGD, AdaGrad, RMSProp, Adam
<code>mynn/lr_scheduler.py</code>	StepLR, MultiStepLR, ExponentialLR
<code>mynn/runner.py</code>	training loop, evaluation, model saving, early stopping, train/eval mode
<code>test_train.py</code>	Part A/B training entry
<code>test_model.py</code>	saved model evaluation on test set
Part C recipe script	optimization and regularization experiments
Analysis script	error analysis and visualization

表 1: Main code files and their responsibilities.

主要复现实验命令如下：

```
cd codes
python test_train.py --model mlp
python test_train.py --model cnn
python test_model.py --model mlp
python test_model.py --model cnn
python experiment_part_c_recipe.py \
    --recipes sgd,momentum,sgd_step,adam,l2,dropout,recipe_combo \
    --epochs 5 --eval-batch-size 1000 \
    --out-dir part_c_results/recipe_full
python analysis_visualization.py
```

1.4 Submission Links

课程要求在最终 PDF 中提供代码仓库链接和训练模型/权重链接。对应链接如下：

- Code repository: GitHub repository
- Trained checkpoints: ModelScope checkpoint repository

2 Part A: MLP Baseline

2.1 Model Definition

MLP 将输入图像展平为 784-维向量：

$$x \in \mathbb{R}^{28 \times 28} \mapsto \text{vec}(x) \in \mathbb{R}^{784}.$$

本实验使用两层隐藏层：

$$784 \rightarrow 32 \rightarrow 16 \rightarrow 10.$$

对第 l 层，线性变换定义为

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)},$$

其中 $A^{(0)} = X$, $W^{(l)} \in \mathbb{R}^{d_{l-1} \times d_l}$, $b^{(l)} \in \mathbb{R}^{1 \times d_l}$ 。隐藏层使用 ReLU:

$$A^{(l)} = \text{ReLU}(Z^{(l)}) = \max(0, Z^{(l)}).$$

最后一层输出 logits $z \in \mathbb{R}^{10}$, 用于分类。

2.2 Softmax Cross Entropy

对一个样本的 logits z , softmax 概率为

$$p_k = \frac{\exp(z_k - \max_j z_j)}{\sum_{r=1}^{10} \exp(z_r - \max_j z_j)}.$$

减去 $\max_j z_j$ 不改变 softmax 结果，但能提高数值稳定性。给定真实标签 y , cross entropy loss 为

$$\ell(z, y) = -\log p_y.$$

对 batch size 为 N 的 mini-batch, 平均损失为

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{i, y_i}.$$

softmax 和 cross entropy 合并后的梯度为

$$\frac{\partial L}{\partial z_{i,k}} = \frac{1}{N} (p_{i,k} - \mathbb{I}[y_i = k]).$$

2.3 Backpropagation of Linear and ReLU Layers

设线性层输出为 $Y = XW + b$ ，下一层传回梯度 $G = \frac{\partial L}{\partial Y}$ 。则线性层反向传播为

$$\frac{\partial L}{\partial W} = X^\top G, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^N G_i, \quad \frac{\partial L}{\partial X} = GW^\top.$$

ReLU 的梯度为

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \mathbb{I}[Z > 0].$$

对应实现位于 `mynn/op.py` 中的 `Linear.forward`、`Linear.backward`、`ReLU.backward` 和 `MultiCrossEntropyLoss.backward`。模型结构由 `mynn/models.py` 中的 `Model_MLP` 组织，训练过程由 `mynn/runner.py` 负责。

2.4 Training Setting

Item	Setting
Model	MLP
Architecture	$784 \rightarrow 32 \rightarrow 16 \rightarrow 10$
Activation	ReLU
Initialization	He-style scaling for ReLU hidden layers
Loss	Softmax cross entropy
Optimizer	SGD
Learning rate	0.1
Batch size	128
Epochs	5
Train/validation split	50000 / 10000

表 2: Part A MLP baseline setting.

2.5 Results and Analysis

Metric	Accuracy
Best validation accuracy	0.94790
Test accuracy	0.95440

表 3: Part A MLP baseline results.

MLP baseline 能够在 MNIST 上达到 0.95440 的 test accuracy，说明手写线性层、softmax cross entropy 和反向传播实现是有效的。但 MLP 的主要限制也很明显：它把 28×28 图像展平成一维向量，打破了图像局部空间结构。相邻像素之间的关系、局部笔画模式和平移等价性都需要模型从数据中重新学习，因此参数利用效率不如 CNN。

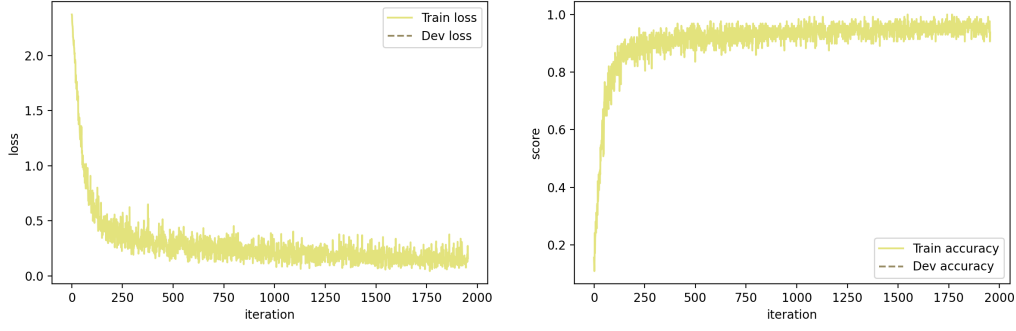


图 1: Learning curve of the MLP baseline.

3 Part B: CNN Model and MLP-vs-CNN Comparison

3.1 Convolution Definition

CNN 的核心是二维卷积。输入记为 $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ ，卷积核为

$$K \in \mathbb{R}^{C_{out} \times C_{in} \times R \times S},$$

bias 为 $b \in \mathbb{R}^{C_{out}}$ 。在 stride 为 t 、padding 后输入为 $\tilde{X}$ 时，输出为

$$Y_{n,o,i,j} = b_o + \sum_{c=1}^{C_{in}} \sum_{r=1}^R \sum_{s=1}^S K_{o,c,r,s} \tilde{X}_{n,c,it+r,jt+s}.$$

本项目中的卷积实现没有调用深度学习框架，而是在 `mynn/op.py` 的 `conv2D` 中使用 NumPy 的 `sliding window view` 构造局部感受野，并用 `einsum` 完成前向卷积。

3.2 Convolution Backpropagation

令 $G = \frac{\partial L}{\partial Y}$ 。卷积核梯度为

$$\frac{\partial L}{\partial K_{o,c,r,s}} = \sum_{n,i,j} G_{n,o,i,j} \tilde{X}_{n,c,it+r,jt+s}.$$

bias 梯度为

$$\frac{\partial L}{\partial b_o} = \sum_{n,i,j} G_{n,o,i,j}.$$

输入梯度则将每个输出位置的梯度按照对应卷积核权重散射回输入窗口。实现中显式遍历 kernel 的空间维度 (r, s) ，并累加到 padded input gradient；若 padding 非零，再裁剪回原输入大小。该实现重点是正确性和可解释性，而不是最高速度。

3.3 Pooling and CNN Architecture

Max pooling 定义为

$$Y_{n,c,i,j} = \max_{0 \leq r < P, 0 \leq s < P} X_{n,c,Pi+r,Pj+s}.$$

反向传播时，梯度只传给最大值所在位置；若有多个相同最大值，代码中将梯度平均分配给这些位置。

CNN 结构为

Conv($1 \rightarrow 4, 3 \times 3, \text{padding} = 1$) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
 $\rightarrow$ Conv($4 \rightarrow 8, 3 \times 3, \text{padding} = 1$) $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ Linear($8 \cdot 14 \cdot 14 \rightarrow 10$).

这一结构保留两层卷积以便观察中间特征，同时通道数较小，适合手写 NumPy 实现在 CPU 上训练。

3.4 Training Setting

Item	Setting
Model	CNN
Architecture	Conv-ReLU-Pool-Conv-ReLU-Flatten-Linear
Conv channels	4, 8
Kernel size	3×3
Padding	1
Pooling	2×2 max pooling
Loss	Softmax cross entropy
Optimizer	SGD
Learning rate	0.05
Batch size	128
Epochs	5
Train/validation split	50000 / 10000

表 4: Part B CNN setting.

为了保持公平，CNN 与 MLP 使用相同的 train/validation split、相同随机种子、相同 batch size 和相同 epoch 数。二者的 learning rate 不完全相同，因为两个模型的梯度尺度和结构不同；但二者均使用固定 learning rate 的 SGD，不引入 Part C 中的 momentum、dropout 或 scheduler。

3.5 MLP-vs-CNN Results

Model	Best validation accuracy	Test accuracy
MLP baseline	0.94790	0.95440
CNN baseline	0.97570	0.97790

表 5: MLP-vs-CNN comparison under similar training settings.

CNN baseline 的 test accuracy 比 MLP 提高了 $0.97790 - 0.95440 = 0.02350$ ，测试错误数也从 456 降到 221。CNN 更适合图像分类的原因主要有三点。第一，局部连接使模型天然关注

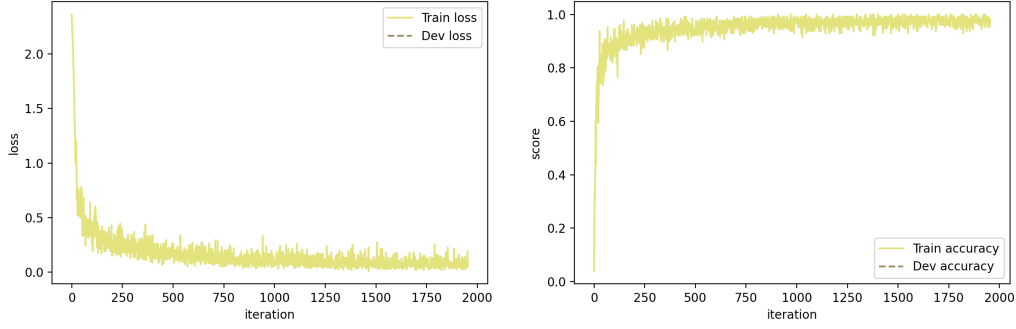


图 2: Learning curve of the CNN baseline.

局部笔画、边缘和角点。第二，权重共享使同一个卷积核可以在不同位置检测相同局部模式，从而提高参数效率。第三，pooling 提供了一定的局部平移稳定性。相比之下，MLP 需要在全连接权重中分别学习不同位置的相似结构。

4 Part C Direction 1: Optimization

4.1 Goal and Experimental Principle

本方向研究同一个 CNN baseline 在不同优化策略下的收敛速度、验证性能和测试泛化。为遵守课程文档中“try to change one major factor at a time”的原则，除最后的组合实验外，每个实验只改变一个主要因素。所有实验固定 CNN 架构、数据划分、随机种子、batch size 和 epoch 数。

4.2 Optimization Methods

SGD 的基本更新为

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L_t.$$

在代码中， θ 对应各层的 `params`，梯度对应 `grads`。实现位于 `mynn/optimizer.py` 的 `SGD.step`。

Momentum SGD 引入速度变量 v_t ：

$$v_{t+1} = \mu v_t - \eta \nabla_{\theta} L_t, \quad \theta_{t+1} = \theta_t + v_{t+1}.$$

其中 $\mu = 0.9$ 。Momentum 利用历史梯度方向，能在一致下降方向上加速，并减小震荡。

Adam 使用一阶矩和二阶矩估计：

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \end{aligned}$$

代码中还实现了 AdaGrad 和 RMSProp，作为经典优化器补充。

Learning rate scheduler 控制 η_t 。StepLR 每隔固定步数乘以 γ ，MultiStepLR 在指定 milestone 衰减，ExponentialLR 每步执行

$$\eta_{t+1} = \gamma\eta_t.$$

对应实现位于 `mynn/lr_scheduler.py`。

5 Part C Direction 2: Regularization

5.1 Goal and Experimental Principle

本方向研究 weight decay、dropout 和 early stopping 对同一个 CNN baseline 的泛化影响。单项实验均以 Momentum CNN 作为 stronger baseline，只改变一个正则化因素；组合 recipe 用于展示这些设置能否共同形成更好的训练方案，但不用于把收益归因到单个技巧。

5.2 Regularization Methods

经典 L2 regularization 对权重加入惩罚项：

$$L_{\text{total}} = L_{\text{CE}} + \frac{\lambda}{2} \|W\|_2^2.$$

等价地，SGD 中可以写作

$$W_{t+1} = (1 - \eta\lambda)W_t - \eta\nabla_W L_{\text{CE}}.$$

本项目在 optimizer 中对 W 使用原地 weight decay，而不作用于 bias。对普通 SGD，这与上式等价；在 Momentum 和 Adam 中，它更准确地说是 optimizer-level weight decay，而不是把 λW 直接加进梯度。

Dropout 在训练时随机屏蔽激活：

$$\tilde{h} = \frac{m \odot h}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p).$$

这是 inverted dropout，因此测试时不需要额外缩放。实现位于 `mynn/op.py` 的 Dropout，并通过 `train/eval` 模式控制训练和验证行为。

Early stopping 以 validation accuracy 为监控指标。如果连续若干次 evaluation 没有提升，则停止训练并保留已保存的最佳模型。本实验的 5 epoch 设置下未提前触发，但该机制被纳入组合 recipe。

5.3 Experiment Settings

实验脚本为 `experiment_part_c_recipe.py`。每个 recipe 保存 best model、learning curve、JSON metrics 和总对比图。完整结果位于 Part C 的 recipe result 目录。

Recipe	Optimizer	Scheduler	Weight decay	Dropout	Early stop
sgd	SGD	none	0	0	no
momentum	Momentum	none	0	0	no
sgd_step	SGD	StepLR	0	0	no
adam	Adam	none	0	0	no
l2	Momentum	none	10^{-4}	0	no
dropout	Momentum	none	0	0.2	no
recipe_combo	Momentum	MultiStepLR	10^{-4}	0.1	yes

表 6: Optimization and regularization recipes.

Recipe	Best val acc	Test acc	Final dev loss	Final lr	Updates
sgd	0.97570	0.97790	0.08697	0.0500	1955
momentum	0.98210	0.98360	0.06439	0.0500	1955
sgd_step	0.97320	0.97390	0.09418	0.0125	1955
adam	0.97760	0.97980	0.07736	0.0010	1955
l2	0.98200	0.98360	0.06466	0.0500	1955
dropout	0.98390	0.98370	0.07265	0.0500	1955
recipe_combo	0.98400	0.98560	0.05481	0.0125	1955

表 7: Part C recipe comparison on the same CNN baseline.

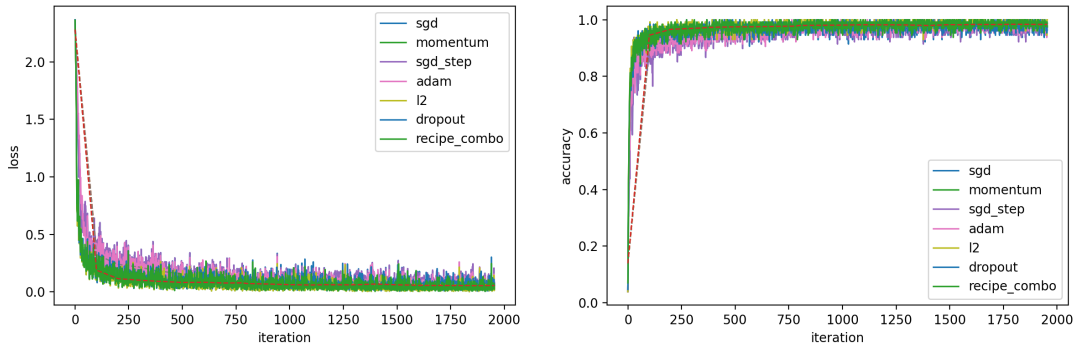


图 3: Learning curves of optimization and regularization recipes. Solid lines are training curves; dashed points are validation curves.

5.4 Results

5.5 Analysis

Momentum 是最直接、最稳定的提升来源。相较于 SGD baseline，它将 validation accuracy 从 0.97570 提升到 0.98210，test accuracy 从 0.97790 提升到 0.98360。结合学习曲线可见，Momentum 在早期 epoch 中下降更快，说明历史梯度方向对这个小型 CNN 的优化非常有效。

Adam 相较 SGD 也有提升，但在当前设置下不如 Momentum。可能原因是模型较小，固定 learning rate 的 Momentum 已经足够稳定；Adam 的自适应缩放没有进一步带来明显优势。

StepLR 在当前 milestone 和 γ 下反而低于固定学习率 SGD，test accuracy 为 0.97390。这说明 scheduler 不是无条件有效的技巧；如果学习率过早衰减，模型可能在后期缺少足够大的探索步长。

Weight decay 在 Momentum 基础上没有明显进一步提升，说明当前 CNN baseline 的过拟合并不严重。Dropout 的 validation accuracy 达到 0.98390，是单项实验中最高的 validation result，但 test accuracy 与 Momentum/weight decay 接近。组合 recipe 达到最佳 test accuracy 0.98560，但它同时改变多个因素，因此只能说明这些技巧共同构成了更好的 training recipe，不能把提升归因于其中某一项。

6 Detailed Visualization and Error Analysis

6.1 Definitions

对测试集预测 $\hat{y}_i$ ，confusion matrix 定义为

$$C_{a,b} = \#\{i : y_i = a, \hat{y}_i = b\}.$$

第 a 类的 class accuracy 为

$$\text{Acc}_a = \frac{C_{a,a}}{\sum_b C_{a,b}}.$$

错分类样本定义为 $\{i : y_i \neq \hat{y}_i\}$ 。为了观察模型最自信但错误的情况，脚本按预测概率

$$\max_k p_{i,k}$$

对错分类样本排序，并可视化置信度最高的若干样本。

6.2 Implementation

分析脚本为 `analysis_visualization.py`，主要完成：

- 加载 MLP baseline、CNN baseline 和最佳 recipe CNN。
- 在完整 test set 上计算 predictions、confusion matrix、每类准确率和 top confused pairs。
- 可视化 MLP 第一层权重，将 784-维权重 reshape 为 28×28 。
- 可视化 CNN 第一层卷积核以及第二层卷积核在输入通道上的平均模式。
- 对指定样本逐层前向，保存 `input`、`conv1`、`relu1`、`pool1`、`conv2`、`relu2` feature maps。

6.3 Confusion Matrix Results

Model	Test accuracy	Number of errors	Frequent confusions
MLP	0.95440	456	$9 \rightarrow 4$, $5 \rightarrow 3$, $7 \rightarrow 9$, $8 \rightarrow 3$
CNN	0.97790	221	$7 \rightarrow 2$, $9 \rightarrow 4$, $6 \rightarrow 5$, $2 \rightarrow 1$
recipe_combo CNN	0.98560	144	$4 \rightarrow 9$, $5 \rightarrow 3$, $7 \rightarrow 2$, $8 \rightarrow 0$

表 8: Error counts and frequent confusion pairs on the test set.

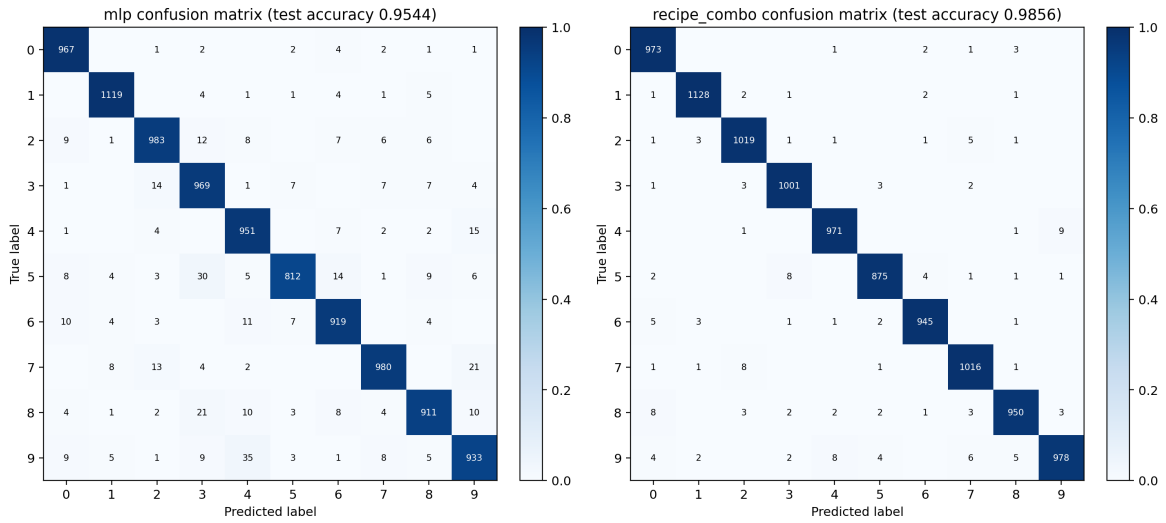


图 4: Confusion matrices of MLP baseline and best recipe CNN.

错误数量从 MLP 的 456 个降到 CNN baseline 的 221 个，再降到最佳 recipe CNN 的 144 个。改进不是平均发生在所有类别上，而是显著减少了很多由局部结构识别不足导致的错误。即便如此，最佳 CNN 仍然容易混淆 $4/9$ 、 $5/3$ 、 $7/2$ 、 $8/0$ 等形状相近数字。这说明 MNIST 中的剩余难例常常具有真实视觉歧义，例如闭环不完整、斜线过长、上半部分弯曲或者局部断笔。

6.4 Weights and Kernels

MLP 第一层权重可视化如下。每个 hidden unit 的 784-维输入权重被 reshape 成 28×28 图像。它们能显示出某些全图区域的正负响应模式，但本质上仍是全图模板，没有显式局部共享结构。

CNN 第一层卷积核只覆盖 3×3 局部区域，更像局部方向、边缘和亮暗变化检测器。第二层卷积核并不是直接的人眼轮廓模板，而是在第一层 feature maps 的不同通道之间组合局部响应。

6.5 Intermediate Feature Maps

为了检验“两层卷积是否真的提取轮廓”，我选取了一个正确样本和一个错误样本，并记录中间 feature maps。设输入图像中像素值大于 0.2 的区域为 foreground，其余为 background。

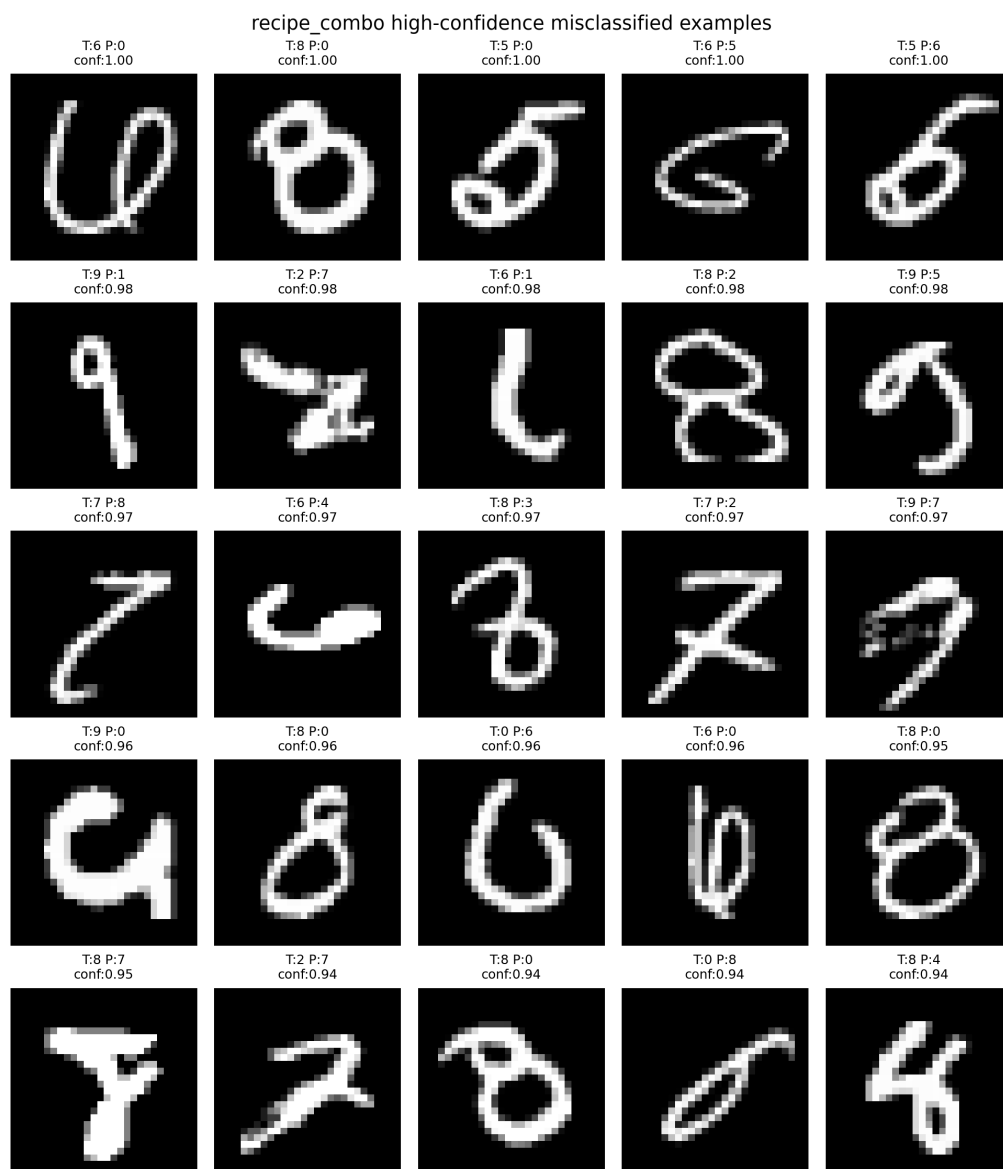


图 5: High-confidence misclassified examples of the best recipe CNN.

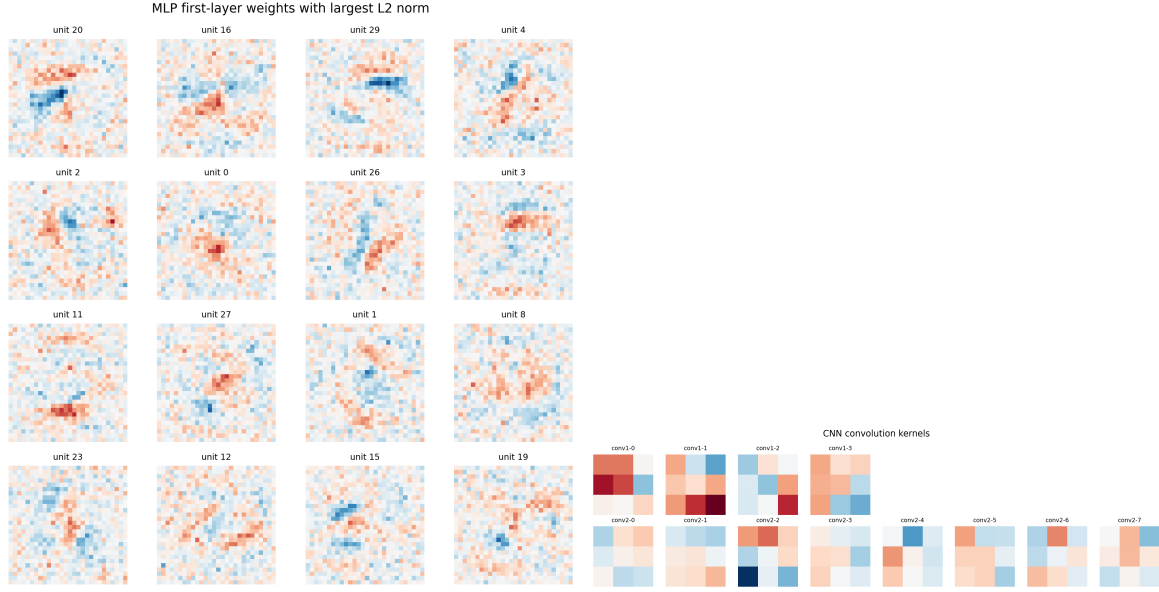


图 6: MLP first-layer weights and CNN convolution kernels.

对某层 feature map F , 定义 foreground/background activation ratio:

$$R = \frac{\text{mean}_{(i,j) \in \text{foreground}} F_{i,j}}{\text{mean}_{(i,j) \in \text{background}} F_{i,j} + \epsilon}.$$

脚本对通道取 ReLU 后平均响应, 再计算该比例。

正确样本 index 3603 的真实标签和预测标签均为 6, 预测置信度接近 1。其 conv1 的 R 约为 9.55, pool1 的 R 约为 11.45, 说明浅层卷积确实显著强化了笔画区域并抑制背景。进入第二层后, conv2 的 R 降到约 2.87, 说明第二层不再只是简单描边, 而是在更低分辨率、更抽象的 feature maps 上组合局部笔画。

错误样本 index 2118 的真实标签为 6, 但被预测为 0, 置信度为 0.9989。它在 conv1 和 pool1 中仍然有较高 foreground response, 但 conv2 的 R 约为 1.95, 低于正确样本。这说明错误并不是因为浅层完全没有检测到笔画, 而是后续组合特征把该样本解释成了更像 0 的结构。

因此, 较严谨的结论是: 第一层卷积和 pooling 后的 feature maps 确实接近笔画/边缘增强; 第二层卷积不等价于输出清晰的人眼轮廓图, 而是进一步组合局部 stroke patterns, 例如闭环、弯曲、斜线和局部端点。

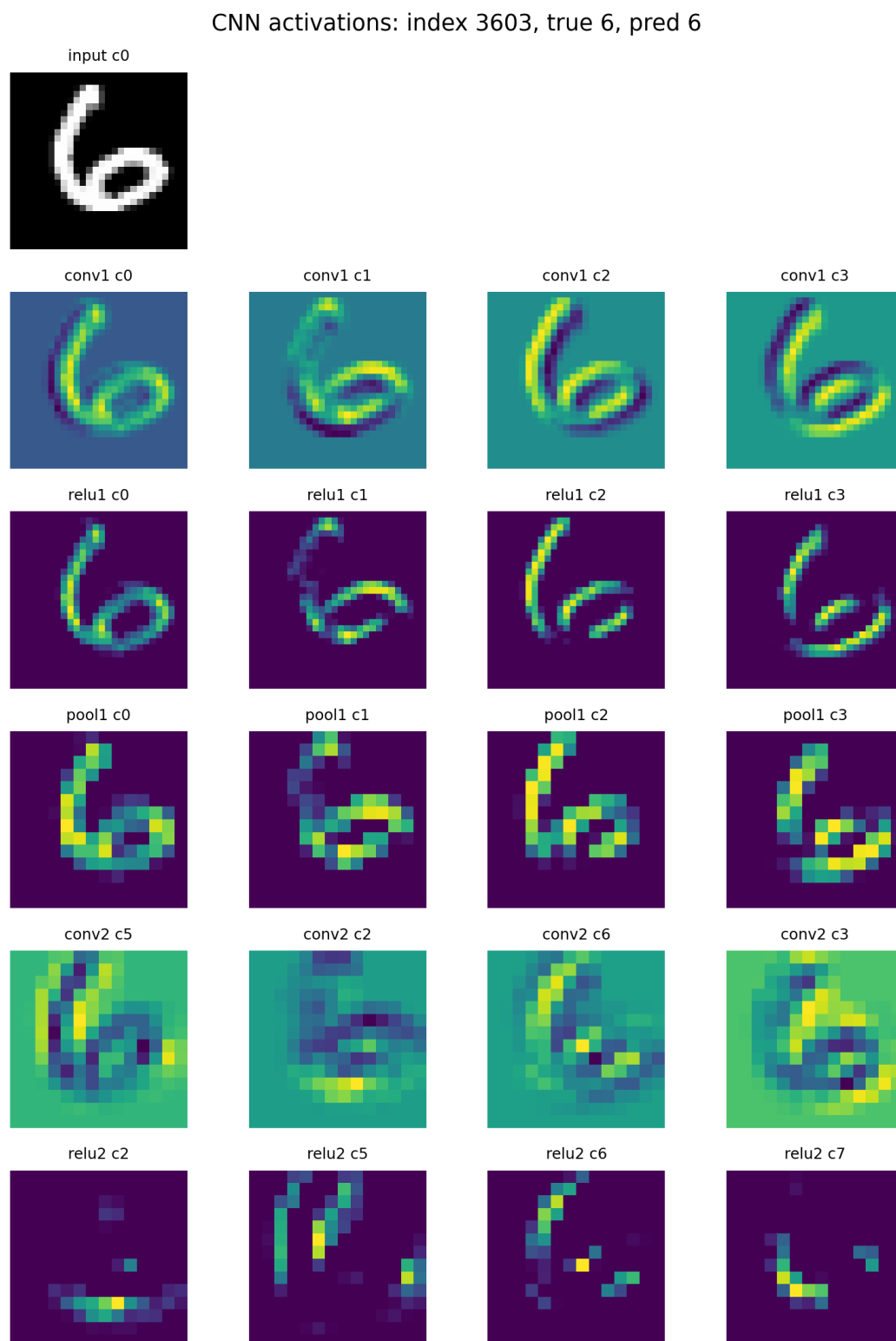


图 7: Intermediate feature maps for a correctly classified digit 6.

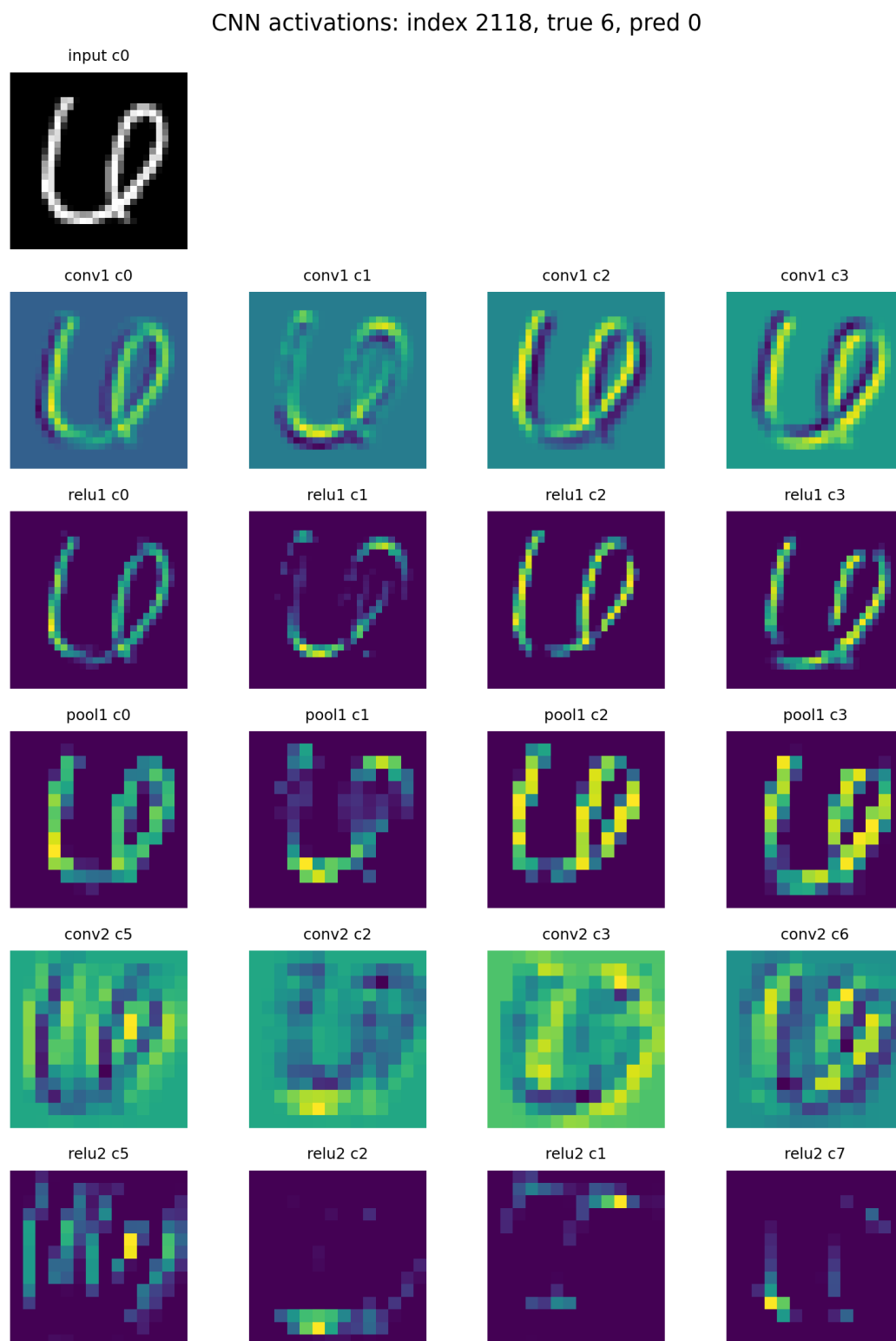


图 8: Intermediate feature maps for a digit 6 misclassified as 0.

7 Main Results Table

Model / setting	Best val acc	Test acc	Note
MLP baseline	0.94790	0.95440	Required Part A baseline
CNN baseline	0.97570	0.97790	Required Part B CNN with handwritten convolution
CNN + Momentum	0.98210	0.98360	Strongest single optimizer improvement
CNN + Adam	0.97760	0.97980	Better than SGD but weaker than Momentum
CNN + WD	0.98200	0.98360	Similar to Momentum; overfitting not severe
CNN + Dropout	0.98390	0.98370	Best single regularization validation result
Best recipe CNN	0.98400	0.98560	Momentum + MultiStepLR + L2 + dropout + early stopping

表 9: Main experimental results.

8 Discussion and Conclusion

本项目的主要结论如下。

第一，CNN 比 MLP 更适合 MNIST 图像分类。MLP 将图像展平，丢失局部空间结构；CNN 通过局部连接、权重共享和 pooling 更有效地利用笔画和边缘信息。因此在相同数据划分和相同 epoch 数下，CNN test accuracy 从 MLP 的 0.95440 提升到 0.97790。

第二，在 optimization 方面，Momentum 是最有效的单项改进。它不仅提升最终 test accuracy，也明显加快早期收敛。Adam 在本任务中有一定收益，但不如 Momentum；StepLR 在当前设置下效果较差，说明 learning rate schedule 需要结合具体训练阶段调节，不能机械使用。

第三，在 regularization 方面，L2 和 dropout 的收益相对温和。当前 CNN 较小，且 MNIST 训练数据充足，因此过拟合并不严重。Dropout 对 validation accuracy 有一定帮助，但单独使用时 test accuracy 与 Momentum 接近。组合 recipe 取得最佳 test accuracy 0.98560，但因为同时改变多个因素，结论应表述为“这些设置共同构成了较好的训练 recipe”，而不是某个单一技巧决定性提升。

第四，错误分析显示，剩余困难样本主要来自视觉歧义，而不是随机错误。即使最佳 CNN 仍会混淆 4/9、5/3、7/2 和 8/0。这些样本往往具有闭环、斜线、断笔或局部形状相似的问题。

第五，可视化结果说明，浅层卷积确实会突出笔画和边缘区域；但“第二层卷积提取轮廓”应谨慎理解。第二层 feature maps 更像是对第一层局部响应的组合，不一定形成直观完整的轮廓图。这也符合 CNN 的层级表征思想：浅层偏局部边缘和纹理，高层偏组合模式和类别相关结构。

整体而言，本项目完成了课程要求的 MLP、CNN、两个额外方向、主结果表和详细可视化。所有核心算子均基于 NumPy 手写实现，没有调用深度学习框架的现成神经网络模块。实验结论也遵循了控制变量原则：先建立可靠 baseline，再逐步比较模型结构、优化方法、正则化方法和错误模式。

9 Verification

已完成以下检查：

- `python -m py_compile` 检查所有实验脚本和 `mynn/` 模块。
- `python test_model.py --model mlp` 输出 test accuracy 0.9544。
- `python test_model.py --model cnn` 输出 test accuracy 0.9779。
- `python experiment_part_c_recipe.py` 成功生成 Part C recipe results。
- `python analysis_visualization.py` 成功生成 error analysis figures。
- `latexmk -xelatex -interaction=nonstopmode report.tex` 成功生成本报告 PDF。